

Advanced Algorithms and Parallel Programming: Challenge 1

Implementation of the primality testing algorithm, asymptotic complexity estimation, tests

- Experimental setup:
 - The code was compiled and run on google colab using C++11
 - The code performance was assessed using the google benchmark library
 - The code was tested using the google test library
 - Numbers of 512 bit size were represented with the boost multiprecision library
 - Random numbers were generated using the boost random library
- Tests:
 - I run different tests on the code, checking for the following cases:
 - Known prime numbers
 - Known composite numbers
 - A large known Mersenne prime
 - A large known Mersenne composite
 - Some edge cases
- Design Choices:
 - I implemented three versions of the benchmark test, each one using numbers with a `uint512_t` representation:
 - The first one checks multiple times the same number with a number of bits depending on the benchmark parameter varying from 64 to 512, with the form of $2^N - 1$
 - The second one checks for each iteration a random number with a number of bits depending on the benchmark parameter, varying from 504 to 512
 - The third one checks for each iteration a random number with a number of bits depending on the benchmark parameter varying from 64 to 512

The reason for this three versions is that i wasn't able to assess a complexity which was consistent with different bit sizes.

I was expecting to find $O(n^3)$ where n is the number of bits used to represent the number, because of the power function calls on $p=N-1$ ($\log p * (\log N)^2$ with $\log N = n$, and $\log p = n = \log N$)
The only case where this happened is the case where I was checking numbers between 2^{504} to 2^{512} .

I think the reason is that the number of bits equal to 512 is still too small to have a consistent assessment of the complexity.